

Smart contract audit

SparkDex

Content

Project description	3
Executive summary	3
Reviews	3
Scope	3
Technical analysis and findings	5
Security findings	6
**** Critical	6
*** High	6
** Medium	6
M01 - Owner can frontrun a redeem() call, potentially blocking the user	6
* Low	7
L01 - A large duration value can lock the user.	7
Informational	7
I01 - Gas Savings	7
I02 - Use of Ownable Instead of Ownable2Step	7
I03 - Validations for the constructor parameters	8
Approach and methodology	9



Project description

This project features a tokenomics system built around three key contracts: Dividends.sol, TokenDistributor.sol, and XSPRK.sol. XSPRK.sol introduces xSPRK, an escrowed token derived from SPRK, which users can stake. The Dividends.sol contract is designed to distribute various token dividends to users who stake their xSPRK within it, managing a continuous yield system. TokenDistributor.sol serves as a general-purpose reward distribution contract, enabling the owner to set and distribute multiple tokens as one-off or scheduled rewards. The primary interrelationship lies between XSPRK.sol and Dividends.sol, where xSPRK is allocated to Dividends.sol to earn passive income.

Executive summary

Type	ERC20, Dividends, TokenDistribution
Languages	Solidity
Methods	Architecture Review, Manual Review, Unit Testing, Functional Testing, Automated Review
Documentation	Not provided
Repository	https://github.com/SparkDEX/sparkdex-contracts.git

Reviews

Date	Commit
07/07/25	2051e181dba1f00a42f499481bc85eaef25d76f3

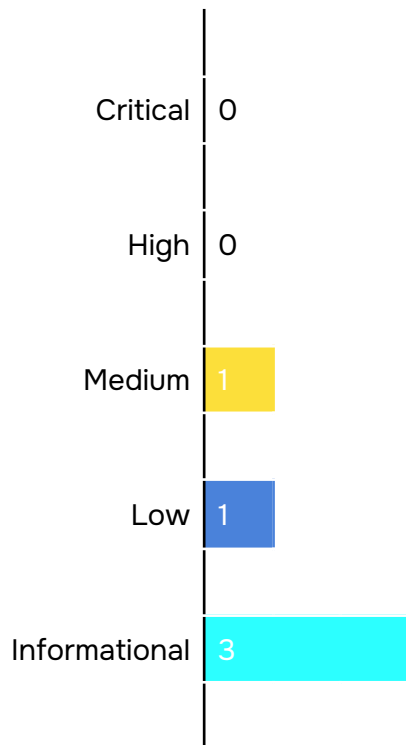
Scope

File Name
contracts/AddressSet.sol
contracts/TokenDistributor.sol
contracts/tokenomics/Dividends.sol
contracts/tokenomics/SPRK.sol



contracts/tokenomics/XSPRK.sol
contracts/tokenomics/lib/Address.sol
contracts/tokenomics/lib/Context.sol
contracts/tokenomics/lib/EnumerableSet.sol
contracts/tokenomics/lib/Math.sol
contracts/tokenomics/lib/Ownable.sol
contracts/tokenomics/lib/ReentrancyGuard.sol
contracts/tokenomics/lib/SafeERC20.sol
contracts/tokenomics/lib/SafeMath.sol

Technical analysis and findings



Security findings

**** Critical

No critical severity issues were identified.

*** High

No high severity issues were identified.

** Medium

M01 - Owner can frontrun a redeem() call, potentially blocking the user

Impact	High
Likelihood	Low

The redeem() function allows users to redeem xSPRK tokens for SPRK tokens through a vesting process. However, there are vulnerabilities where the contract owner can front-run user transactions to manipulate the redeem parameters or conditions:

- with the setWaitingUsersForRedeem() the owner can temporarily disable the redeem() functionality for a specific user;
- with the updateRedeemSettings() the owner can increase the minRedeemDuration to be bigger than the duration input by the user, or decrease the redeemDividendsAdjustment parameter for dividends allocation;
- with the updateDeallocationFee() the owner can update the deallocation fee by increasing the fee right before deallocation transaction.

Such front-running behavior could result in the redeem() function being blocked for a user.

Path: src/contracts/tokenomics/XSPRK.sol

Recommendation: Consider adding a timelock to the updateRedeemSettings(), updateDeallocationFee(), setWaitingUsersForRedeem() functions and also upper and lower time-bounds in order to make the system more predictable and reliable. The timelock for the setWaitingUsersForRedeem() function can likely be activated after the required list of team addresses has been added.

Found in: 2051e18

Status: Fixed at 51fa890



* Low

L01 - A large duration value can lock the user.

Impact	Low
Likelihood	Low

The `redeem()` function allows initiating the process of redeeming xSPRK tokens for SPRK. The ``duration`` value specified by the user can be extremely large.

If a user mistakenly provides an excessively high value for duration, they may be locked for a significant period of time.

Path: `src/contracts/tokenomics/XSPRK.sol`

Recommendation: Consider adding validation to prevent excessively large ``duration`` values.

Found in: 2051e18

Status: Fixed at c5ed87c

Informational

I01 - Gas Savings

In the Dividends contract:

- 1) the ``_cycleDurationSeconds`` variable can be declared as constant.
- 2) the ``onlyOwner()`` modifier of the `emergencyWithdrawAll()` function can be removed, because the ownership is checked in each `emergencyWithdraw()` function call.

Path: `src/contracts/tokenomics/Dividends.sol`

Found in: 2051e18

Status: Fixed at c5ed87c



I02 - Use of Ownable Instead of Ownable2Step

Multiple contracts in the SparkDEX protocol use the standard Ownable pattern instead of Ownable2Step, which creates a security vulnerability where ownership can be permanently lost if the owner address is set to an incorrect or compromised address.

Path: src/contracts/tokenomics/XSPRK.sol, src/contracts/tokenomics/Dividends.sol, src/contracts/TokenDistributor.sol

Recommendation: Consider using OZ's Ownable2Step contract.

Found in: 2051e18

Status: Fixed at c5ed87c

I03 - Validations for the constructor parameters

Some constructor parameters in the Dividends and XSPRKToken contracts are not validated. This may lead to incorrect contract initialization and the need for a subsequent redeployment.

Path: src/contracts/tokenomics/XSPRK.sol, src/contracts/tokenomics/Dividends.sol.

Recommendation: Add validation for the sprkToken_ parameter in the XSPRKToken contract, and for the startTime_ and weth_ parameters in the Dividends contract.

Found in: 2051e18

Status: Fixed at c5ed87c



Approach and methodology

To establish a uniform evaluation, we define the following terminology in accordance with the OWASP Risk Rating

Methodology:

	Likelihood Indicates the probability of a specific vulnerability being discovered and exploited in real-world scenarios
	Impact Measures the technical loss and business repercussions resulting from a successful attack
	Severity Reflects the comprehensive magnitude of the risk, combining both the probability of occurrence (likelihood) and the extent of potential consequences (impact)

Likelihood and impact are divided into three levels: High H, Medium M, and Low L. The severity of a risk is a blend of these two factors, leading to its classification into one of four tiers: Critical, High, Medium, or Low.

When we identify an issue, our approach may include deploying contracts on our private testnet for validation through testing. Where necessary, we might also create a Proof of Concept PoC to demonstrate potential exploitability. In particular, we perform the audit according to the following procedure:

	Advanced DeFi Scrutiny We further review business logics, examine system operations, and place DeFi-related aspects under scrutiny to uncover possible pitfalls and/or bugs
	Semantic Consistency Checks We then manually check the logic of implemented smart contracts and compare with the description in the white paper.
	Security Analysis The process begins with a comprehensive examination of the system to gain a deep understanding of its internal mechanisms, identifying any irregularities and potential weak spots.

